

AI Frameworks 2025 Project Report

Author

PersonRobin Herickx

Table of Contents

Part 1: Project Description and Introduction

Here is the updated and corrected Table of Contents based on the headings in your "AI Frameworks 2025 Project Report" document:

Table of Contents

Part 1: Project Description and Introduction

Part 2: Data Collection and Cleaning

- 2.1 Data collecting
- 2.2 Label Studio
- 2.3 Custom Scripts data extraction
 - 2.3.1 Main Data Extraction Script for YOLO
 - 2.3.2 Personal Data Extraction Script
 - 2.3.3 Face Extraction
 - 2.3.4 YOLO Viewer
 - 2.3.5 YOLO Editor
- 2.5 Final Code

Part 3: YOLO (You Only Look Once)

- 3.1 What is YOLO? A Quick Overview
- 3.2 Training Different Models and Methods
- 3.3 YOLO Result
 - 3.3.1 yolo11 L (no augmentations)
 - 3.3.2 yolo11 X (augmentations)
 - 3.3.3 The Rayen set (yolo 11 X augmentation)
 - 3.3.4 yolo face detection (yolo 11 Nano)

Part 4: Face Detection

- 4.1 How it works
- 4.2 DeepFace
- 4.3 InsightFace and buffalo_I
- 4.4 Face dataset
- 4.5 Embedding and clustering

- 4.5.1 Unsupervised learning
 - 4.5.2 Supervised learning (my approach)
- 4.6 experimentation
- 4.7 Embedding and Clustering Results
 - 4.7.1 PCA (Principal Component Analysis)
 - 4.7.2 t-SNE
 - 4.7.3 UMAP
 - 4.7.4 Conclusion

Part 5 – Inference Code

- 5.1 Inference Pipeline
 - 5.1.1 Logging and CSV export
 - 5.1.2 Find more faces
- 5.2 Cosine Similarity
- 5.3 Combining Different Methods
- 5.4 Combined Methods Results

Part 6 – GUI and Masking Clarification

- 6.1 Gradio
- 6.2 Segment People with SAM3
- 6.3 SAM3 Implementation and Pipeline
 - 6.3.1 Code Explanation
- 6.4 Results and GUI Usage

Part 7: Conclusion and Future Work

- **Appendix: Relevant Files**

Part 1: Project Description and Introduction

This document outlines the scope methods and initial findings of the AI Frameworks 2025 project. The goal is to explore and implement various Artificial Intelligence frameworks for person classification. For example given an image of our classmates we want to label all the people shown from left to right. As an example Image X as input gets the string “Robin Ennis Rayen” as output.

This document will show what I have done to get to the end result.

First I will go through the project timeline from the beginning detailing what I have done and attempted.

Starting from data collection and data cleaning.

Then I will detail how I used YOLO for full-body classification and all the different methods I have tried.

Next I will go into face detection using the InsightFace library.

In Part 5 I will go deeper into what I have tried for image processing and what the results were.

Part 6 is about making a user-friendly interface (GUI) using Gradio and SAM for person segmentation.

Then in Part 7 and 8 I will show the final project layout and what I have learned.

Part 2: Data Collection and Cleaning

The integrity of any AI project hinges on the quality of its data. This section details the steps taken to collect label and clean the data

2.1 Data collecting

We took **454 pictures** in total of which **62 were in the test set** and **392 images in the training set**.

There are also **11 videos** for my project; I mainly focused on the videos. I selected **9 videos** to use (some videos were too low quality or too short to use).

Then I used **Label Studio** and **custom scripts** for data extraction to make **3 datasets** for training.

There are about **3200 samples** in each data set .

Dataset 1:

All people labeled in **YOLO format**.

YOLO format per line:

`<class_id> <x_center> <y_center> <width> <height>`

(all values normalized between 0 and 1).

The dataset has **2 folders**:

- `images/` → all image files
- `labels/` → YOLO `.txt` files

Each label file has the **same base name** as its corresponding image (e.g. `frame_001.jpg` ↔ `frame_001.txt`).

Dataset 2:

This is the same as Dataset 1 but I rewrote the script so that **each person has their own folder**.

This was originally to train different models for each person. I later found out that this was unnecessary for YOLO because during training I can add a **filter for the object/class to train on**.

Dataset 3:

This dataset is for **faces**. It has the same layout as Dataset 2: **one folder per person** and inside that folder all the **face images** of that person.

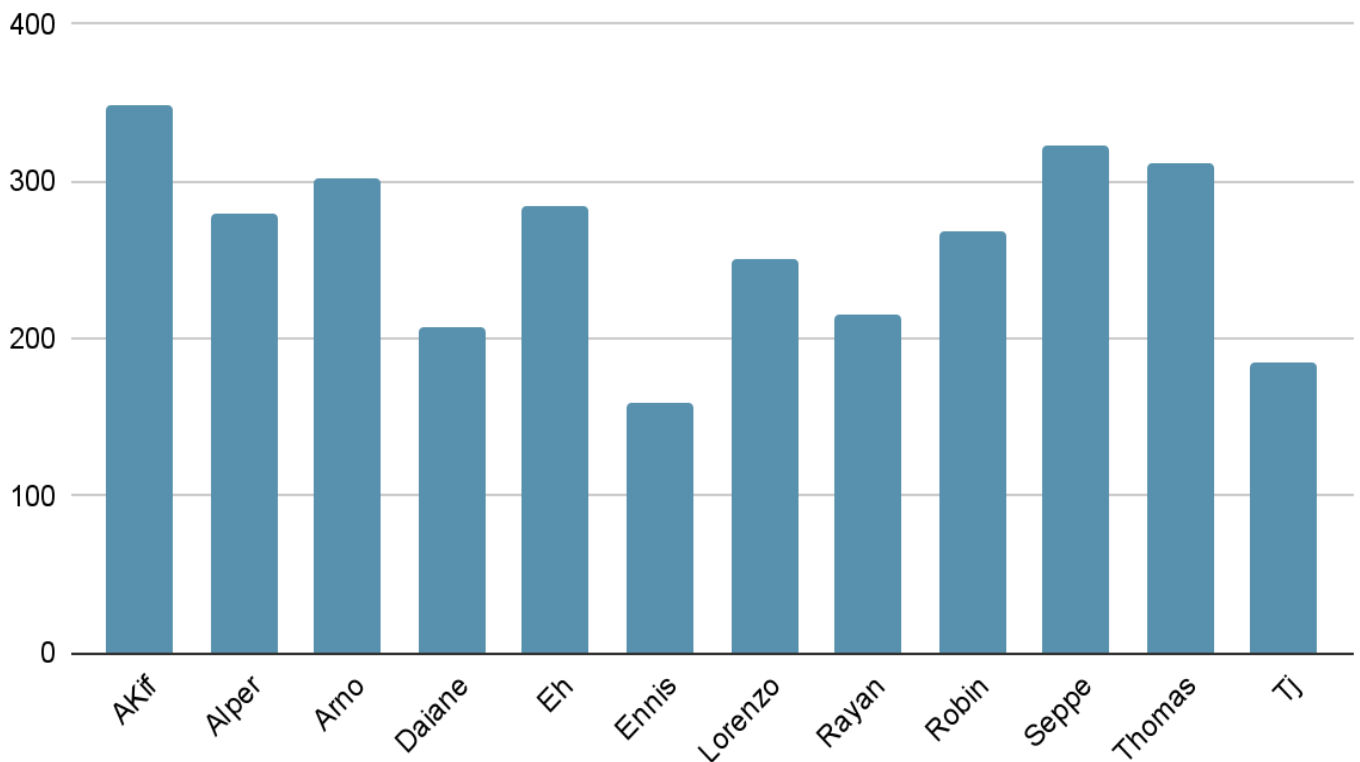
Then I noticed there was an **imbalance** in the amount of people in the dataset (for example **Akif** appeared in way more videos/frames than **Diane** for every Diane there were about **6 Akifs**).

To fix this I manually selected and added data to the dataset. Using the training data I did **not remove people**

Looking back at it now it is still not that well-balanced but I wanted to start training and thought it was good enough. The max deviation originally was 6:1 (349 images for akif and 60 for Diane). Now the max deviation is 2:1 (from 349 Akif to 159 Enis) which for the models I used in the end does not really matter but could have added even more data points for Enis Diane and Tj.

This was the **final result**.

data points



2.2 Label Studio

Label Studio was the primary tool used for data annotation. This involved defining bounding boxes and labeling each person in the video. I did this using the video annotation tool in Label Studio. I labeled keyframes; for example I labeled Frame A and then Frame B twenty frames later. When exporting from Label Studio the labeled video data is in JSON format (the JSON format contained the bounding box coordinates and labels for the keyframes). I could not use this directly so I created a script to fix this. I put the JSON file and the video into my script.

The output was two maps that I could use to train a YOLO model: image files for the frames and corresponding `.txt` files for the labels.

The script also files in between frames using interpolation

2.3 Custom Scripts data extraction

I will not go into too much detail how my cleaning scripts work (line by line) else this document will be too long but I will go over all the scripts I made, what they do, how they work, and the main functions they use. In total, I made 5 scripts for data extraction and cleaning, of which I used 3.

2.3.1 Main Data Extraction Script for YOLO

The goal of this script is to process the raw JSON annotation file from Label Studio into a format suitable for training a YOLO model. This involves converting the JSON data and generating interpolated data between annotated keyframes.

The script's workflow is as follows:

1. Start

When the script starts, it prompts the user to provide the following parameters:

- A file path to a directory containing the videos.
- A file path to a directory containing the corresponding JSON annotation files.
- The **interpolation amount** (a parameter that controls the density of generated frames between keyframes).
- A path to an **output folder**.

If the output folder does not exist, the script will create it along with two sub-folders:

- **labels/**: To store the YOLO-format **.txt** label files.
- **frames/**: To store the extracted frame images.

2. Compatibility Checks and Processing

The script then begins its main processing loops, starting by iterating over each video in the input folder and checking for a corresponding JSON file.

- **Error Handling:** If a corresponding JSON file is not found, the video is skipped and an error message is generated.
- **Compatibility Check:** The script checks if the video and the JSON file are compatible (if the frame count information within the JSON matches the video). If they are incompatible, the video is also skipped.

3. Data Extraction and Interpolation

The core logic uses nested loops to process the data:

- **The JSON format** contains a list of labels (people), and each label has a list of keyframes, with each keyframe holding a frame number and bounding box coordinates.
- **Nested Loops:** The script uses a main loop for each video, a second loop for each label (person) in that video, and a third loop for all the keyframes of that label.

- **Interpolation:** The third loop takes a keyframe and the subsequent keyframe to interpolate the data using the `interpolation amount` parameter. This process is applied to both the frame ID and the bounding box coordinates.

Example of Interpolation:

If keyframe 1 has `ID=1` and the next keyframe is at `ID=5` and the bounding box for that person is interpolated over the segment the output for that person's segment will be two lists:

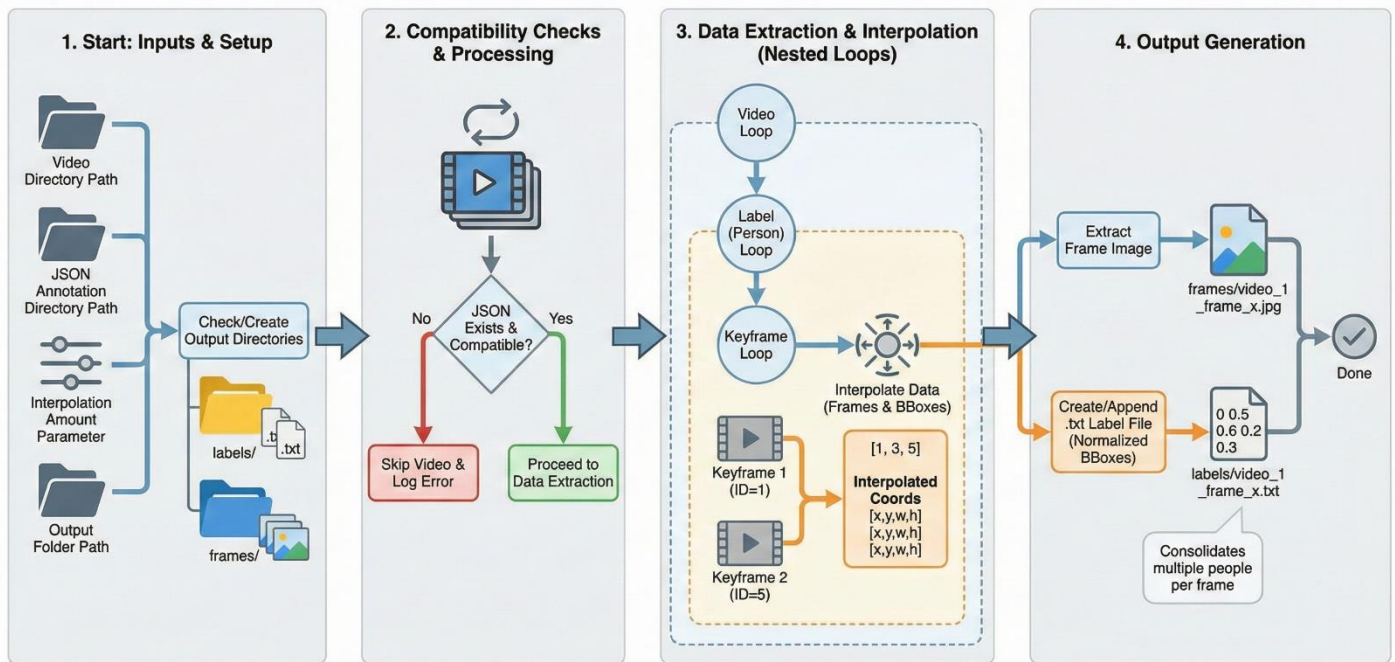
- **Frame IDs:** A list of interpolated frame IDs (`[1, 3, 5]`).
- **Bounding Box Coordinates:** A list of corresponding interpolated bounding boxes. For example if the original bounding boxes were `[[x1,y1,x2,y2] at frame 1]` and `[[X1,Y1,X2,Y2] at frame 5]`, the interpolated list would be `[[1, 1, 5, 5], [6, 6, 10, 10], [11, 11, 16, 16]]`. This process estimates the person's movement between the manually labeled keyframes (note: this interpolation is not always perfect and any resulting bad data will be filtered out in a later script).

4. Output Generation

Once the interpolated Frame ID and Bounding Box lists are generated for a person the script exits the third loop and returns to the second loop to generate the YOLO files:

- For each frame ID the frame is extracted from the video and saved as an image file in the `frames/` folder using a consistent naming convention (`video_1_frame_x.jpg`).
- A corresponding `.txt` label file is created in the `labels/` folder with the same base name (`video_1_frame_x.txt`).
- This file is populated with the label (class ID) and the corresponding normalized bounding box data.
- **Handling Multiple People:** If multiple people appear in the same frame the script **appends** the label and bounding box data for each subsequent person to the *already existing* `.txt` file for that frame ensuring that all people and their bounding boxes for a single frame are consolidated into one label file.

YOLO Main Data Extraction Script Overview



2.3.2 Personal Data Extraction Script

This script is a modification of the main data extraction script (2.3.1) with the key difference being that each person (label) is saved into its own distinct folder.

This approach was originally intended for training a separate YOLO model for each individual person. However I later discovered that this was unnecessary as the YOLO framework already provides a way to filter the training data by the class ID (label) to train on specific objects. Consequently this script was not used for the final YOLO training dataset.

While the code implementation of this script is cleaner(updated version of the original script) its significant drawback is inefficiency due to data duplication:

- If a single frame contains multiple people (Robin and Enis) the image data is stored multiple times once in Robin's sub-folder and once in Enis's sub-folder.
- This inefficiency worsens as the number of people in a frame increases (10 people in one frame would mean the image is saved 10 times).

Because of this inefficiency the original script (2.3.1) was ultimately used for data extraction.

2.3.3 Face Extraction

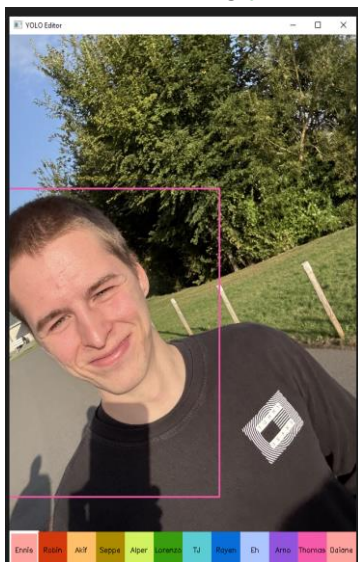
This was a very simple script used to create Dataset 3 (Faces). It uses the previous dataset which includes the bounding box/label and the frame along with the Buffalo L model from Insight Face (more about this in Chapter 4) to extract the faces. Each face is saved as "id_label " for example **1_Robin.png** in a selected output folder. These faces will be embedded later.

2.3.4 YOLO Viewer

To verify if script 2.3.1 worked I needed a script to view the annotated YOLO data. This script takes the path to a dataset as input. It then displays the first image in the dataset and draws the corresponding bounding boxes on it with the correct labels using the corresponding YOLO `.txt` file for that image. Keys "4" and "5" allow you to scroll to the next and previous image. Pressing "8" will move an image and its corresponding `yolo.txt` file to a garbage folder. Pressing "2" will return the last deleted image back to the dataset. This makes viewing and cleaning the generated dataset much easier (but I was not able to edit or add bounding boxes in this version).

2.3.5 YOLO Editor

This is the final version of script 2.3.4. It was rewritten to be much cleaner have a nicer interface and add the important features of adding deleting and editing bounding boxes. Now at the bottom you can see a row of colors. Selecting a color allows you to choose the corresponding label. You can then left-click and drag to create a bounding box of that label. Placing the mouse cursor in a bounding box and pressing the "x" key will remove that bounding box. Placing your mouse in the bounding box and holding down the right mouse button allows you to move/drag that bounding box. Pressing "s" will save your edits and overwrite the corresponding `yolo.txt` file for that image. All the other functions remain the same.



Part 3: YOLO (You Only Look Once)

This part focuses on the application of the YOLO framework for object(person) detection covering training methodologies and specific results obtained from the YOLO models.

3.1 What is YOLO? A Quick Overview

This part focuses on the application of the YOLO framework for object (person) detection covering training methodologies and specific results obtained from the YOLO models.

3.1 What is YOLO? A Quick Overview

YOLO is a state-of-the-art real-time object detection system. It rephrases object detection as a single regression problem directly predicting bounding boxes and class probabilities from full images in one evaluation. It is really good and easy to train.

Here is a quick list of the metrics and what they mean. This is important for the results in 3.3.

Metric	Definition
Box loss	Bounding box position/size accuracy
Cls loss	Classification accuracy
DFL loss	Bounding box edge precision
Precision	Percentage of detections that are correct
Recall	Percentage of real objects that are found
mAP50	detection quality at IoU ≥ 0.50 (lenient))
mAP50–95	Average quality at IoU 0.50–0.95 (strict main metric)
F1-score	Balance between precision and recall
Confidence (conf)	Certainty per detection used as threshold

3.2 Training Different Models and Methods

I tried training many different YOLO models at the beginning of the project. The results were okay but not great. While the validation set seemed very good testing on the test set was worse.

My final YOLOv11 X model got good results but compared to just face detection it was bad.

I tried many models and iterations but I will not go over all of them. I will show the results of my first model (YOLOv11 L no augmentations) and my latest model (YOLOv11 X with augmentations).

The "Rayen" model: I wanted to train a model for each person. I gave up after seeing the results for one person "Rayen"; the results were really bad. After that I started doing face detection and decided it was not worth spending more time on YOLO.

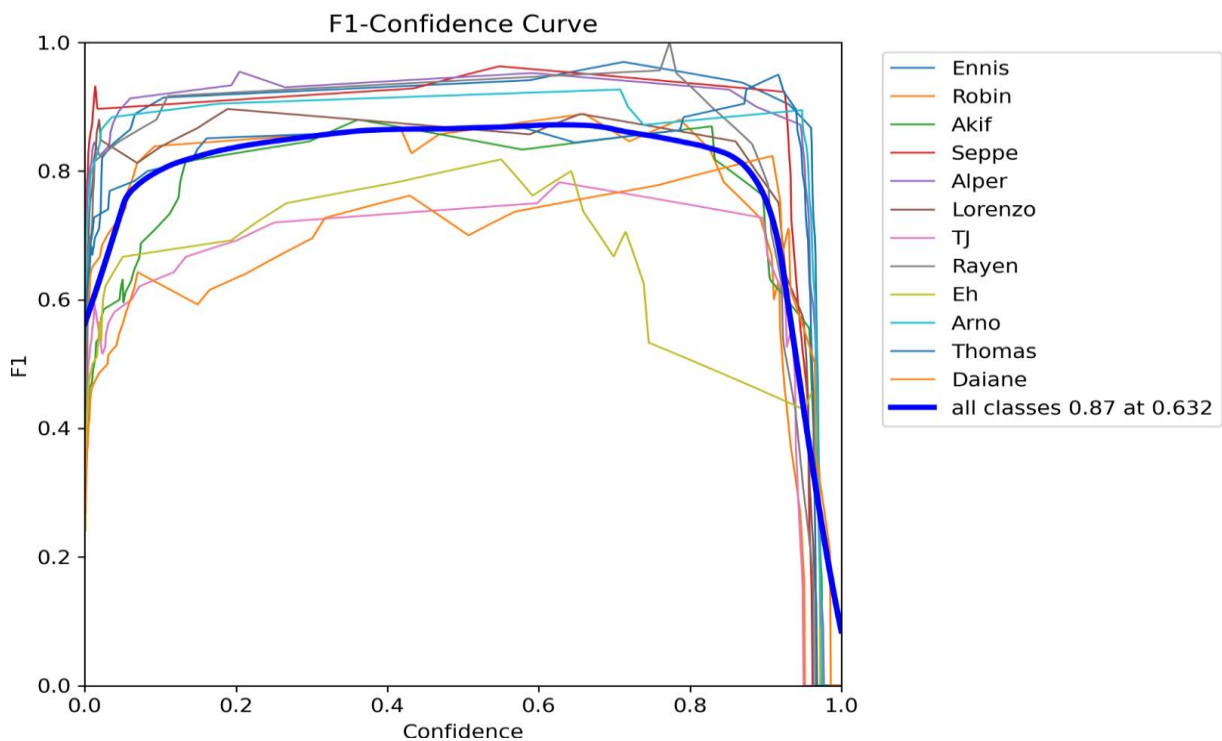
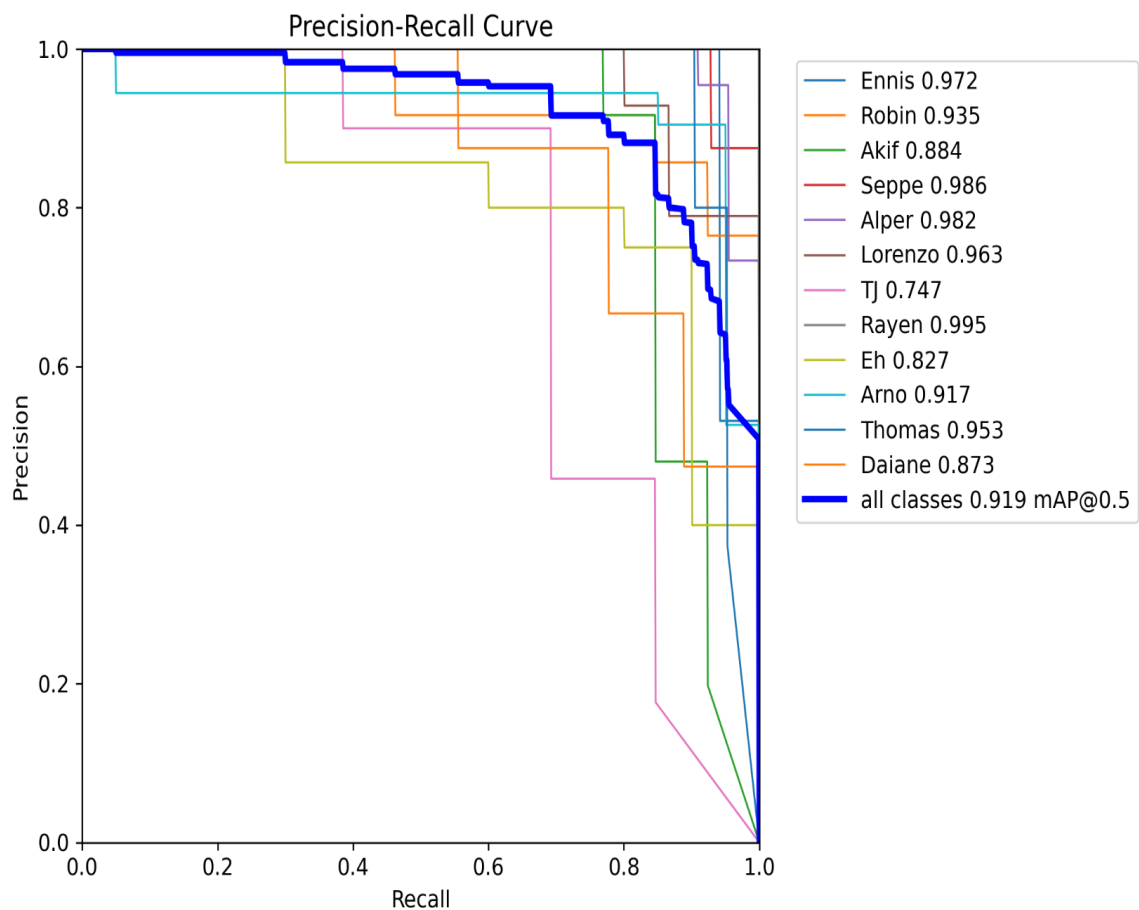
At the end of the project I also made a YOLO faced person model

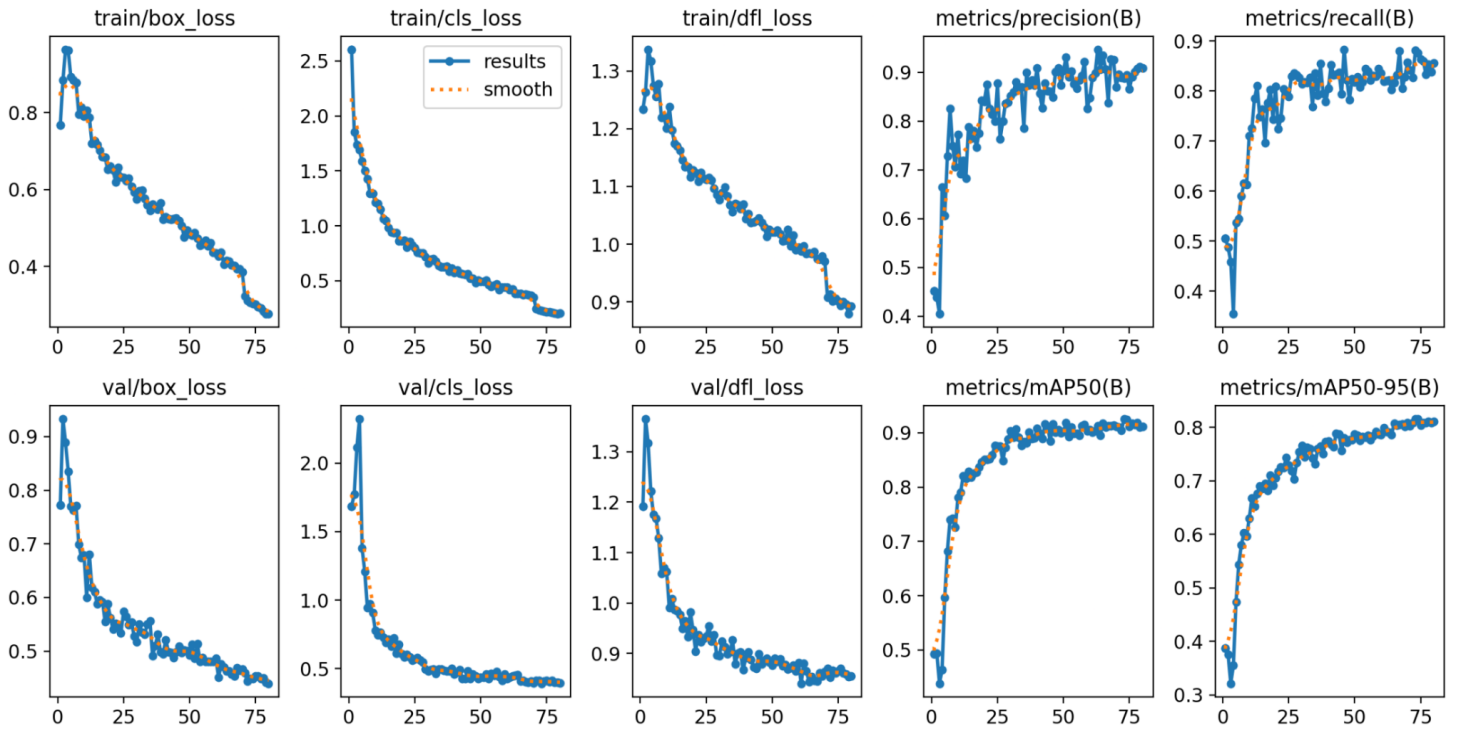
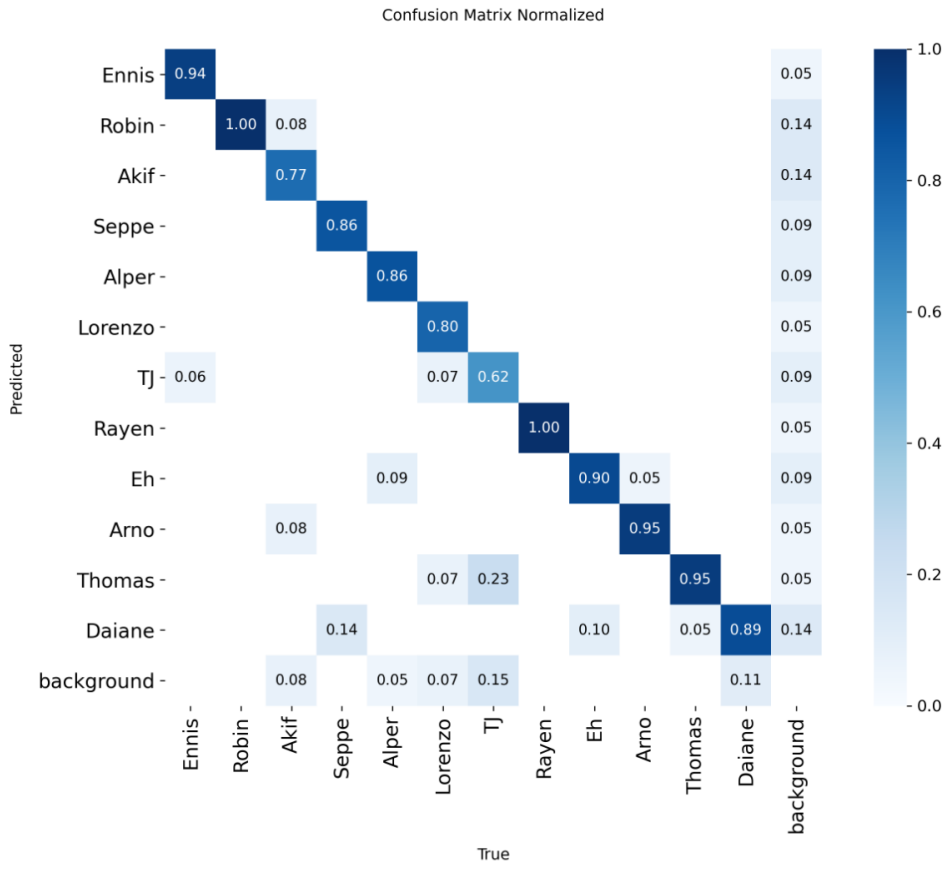
3.3 YOLO Result

Here are the results of these 4 models

3.3.1 yolo11 L (no augmentations)

These are the results/data for one of the first YOLO models I tested YOLO11 L (no augmentations)





Validation Batch prediction:



Conclusion for this model: This first model is not good at all. It is effective when only one person is in the picture but when multiple people are present the model fails and makes many wrong predictions. Note that for the Kaggle dataset one wrong prediction at the beginning of the list will also cause all subsequent predictions to be wrong.

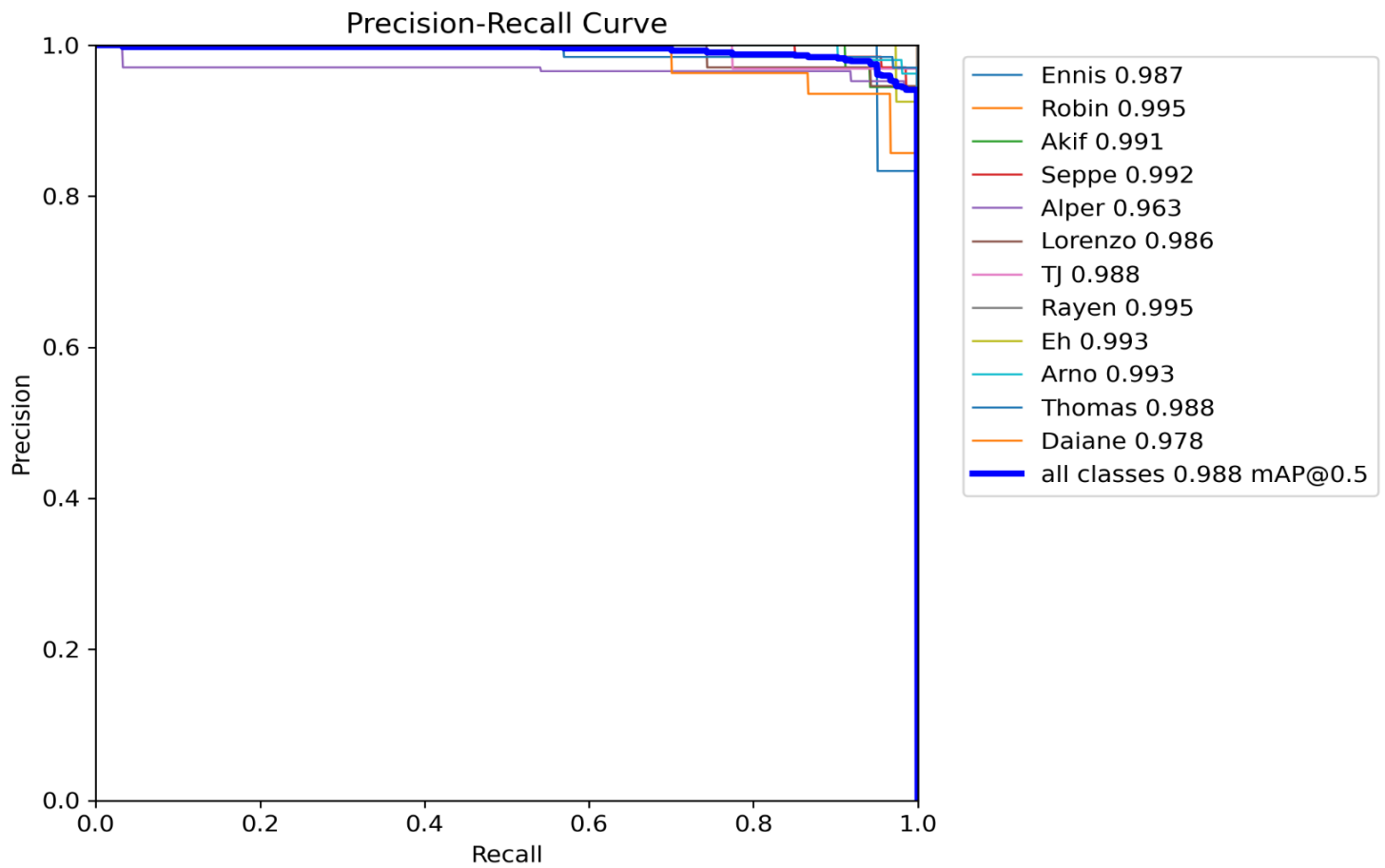
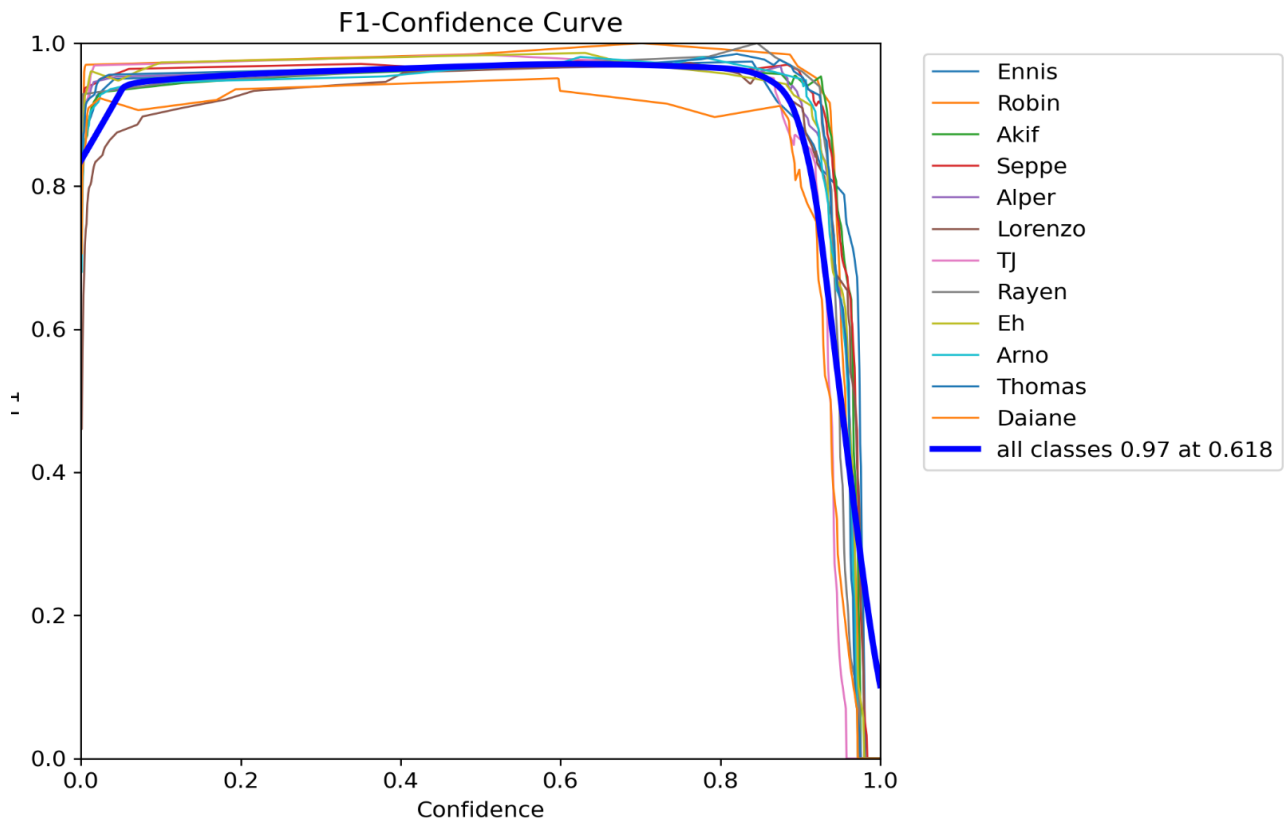
We can also see in the results that we overfitted a bit on the training set compared to the validation set but that is not really the case. YOLO's `best.pt` (the best model we trained so far) is based on the validation set results so if the validation set does not improve the `best.pt` model will not update.

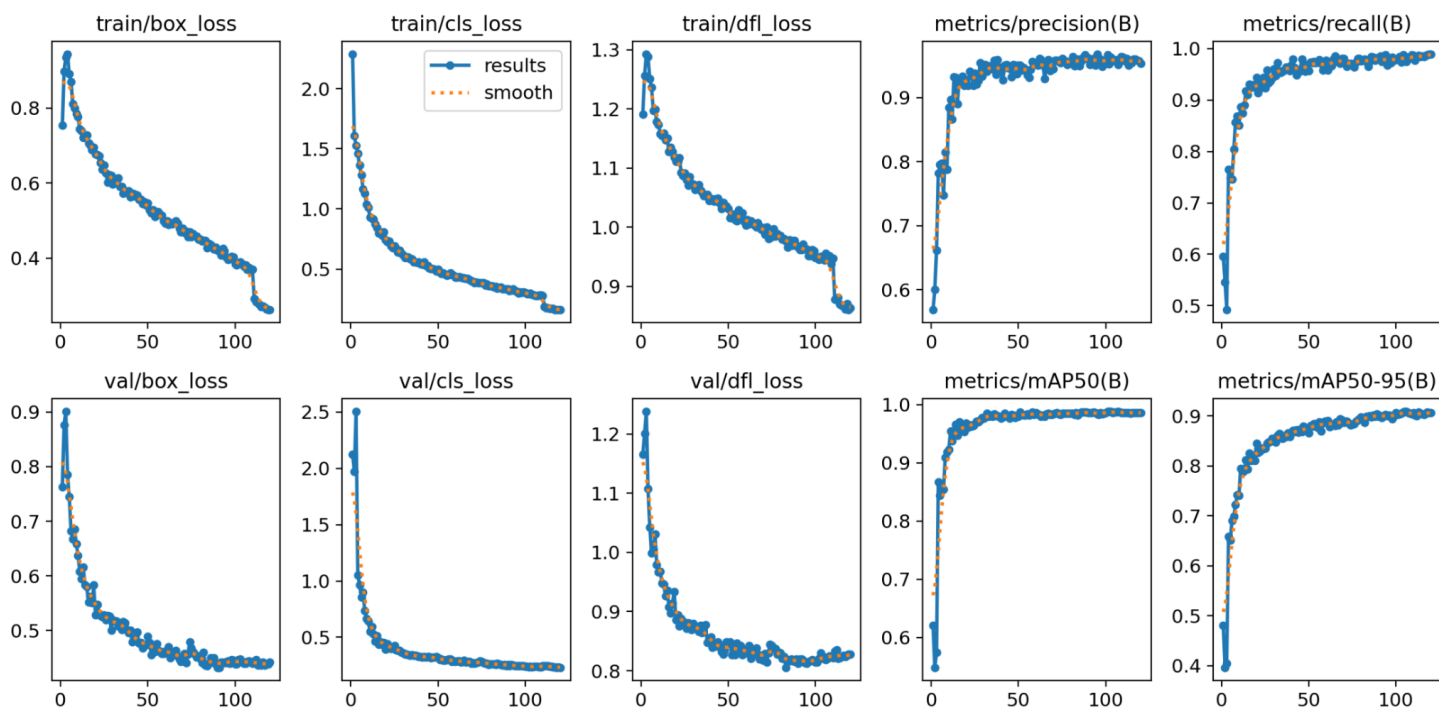
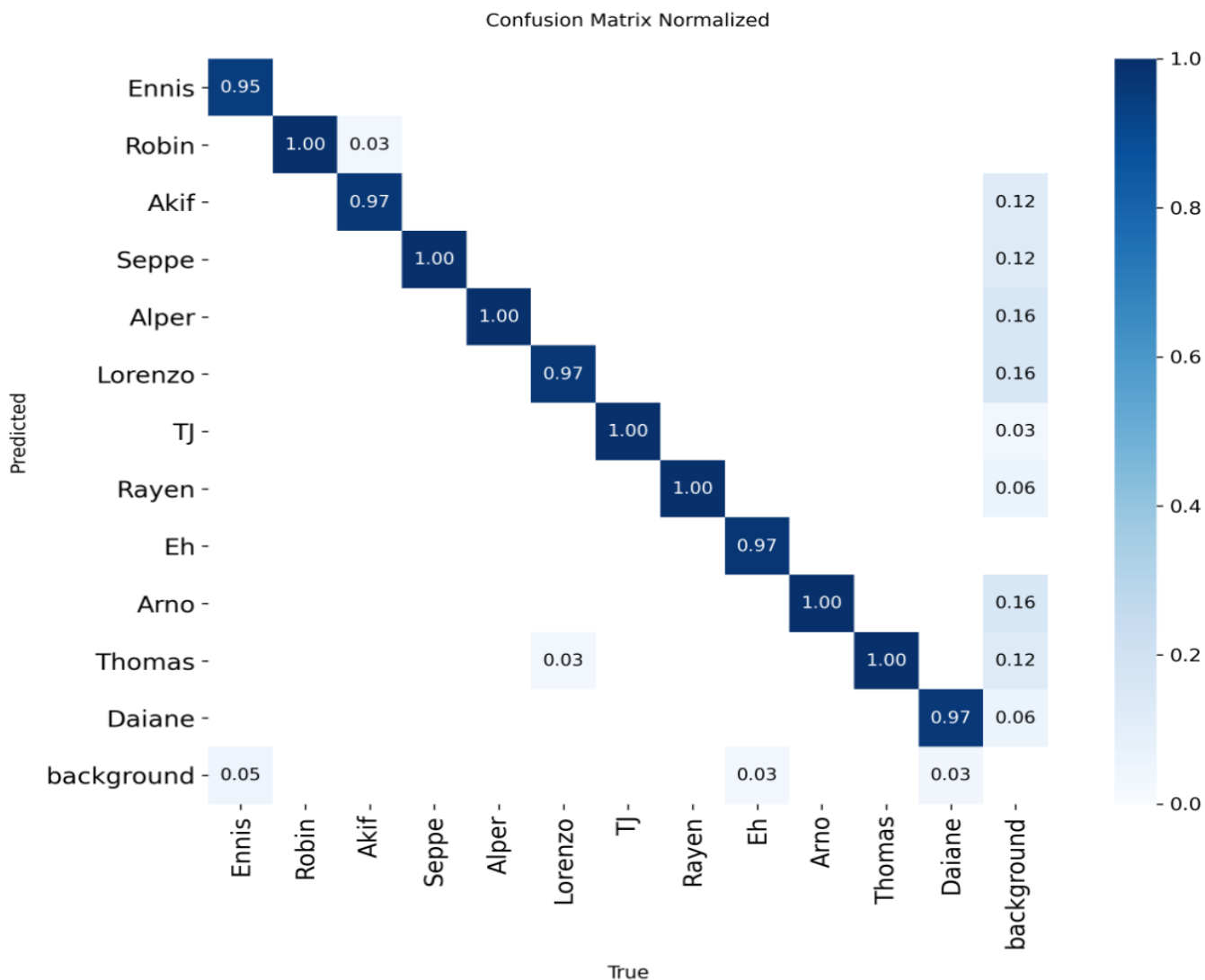
Also early stopping is implemented so if the validation set does not improve after X amount of epochs we stop training.

3.3.2 yolo11 X (augmentations)

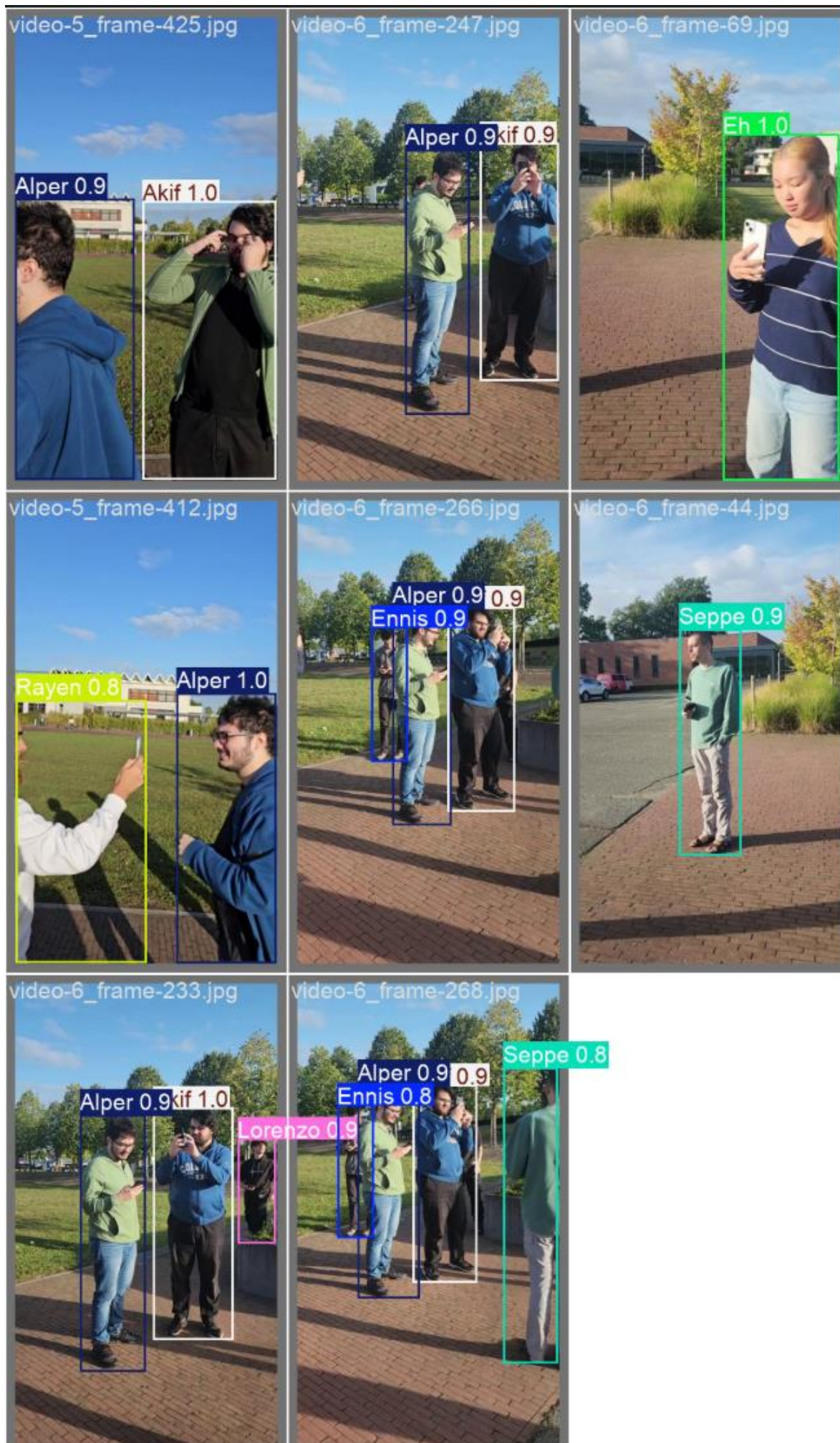
These are the results and data for the last YOLO model that I trained. The improvements include:

1. A better larger and more accurate dataset.
2. A bigger YOLO model YOLO 11_X (11L has 25.3 million parameters and 11X is more than double that with 56.9 million parameters).
3. YOLO image augmentation. I implemented the following augmentations: a horizontal flip with a 40% chance and random contrast adjustment (to learn better in all lighting conditions).
4. A higher early stopping threshold so we train for longer.





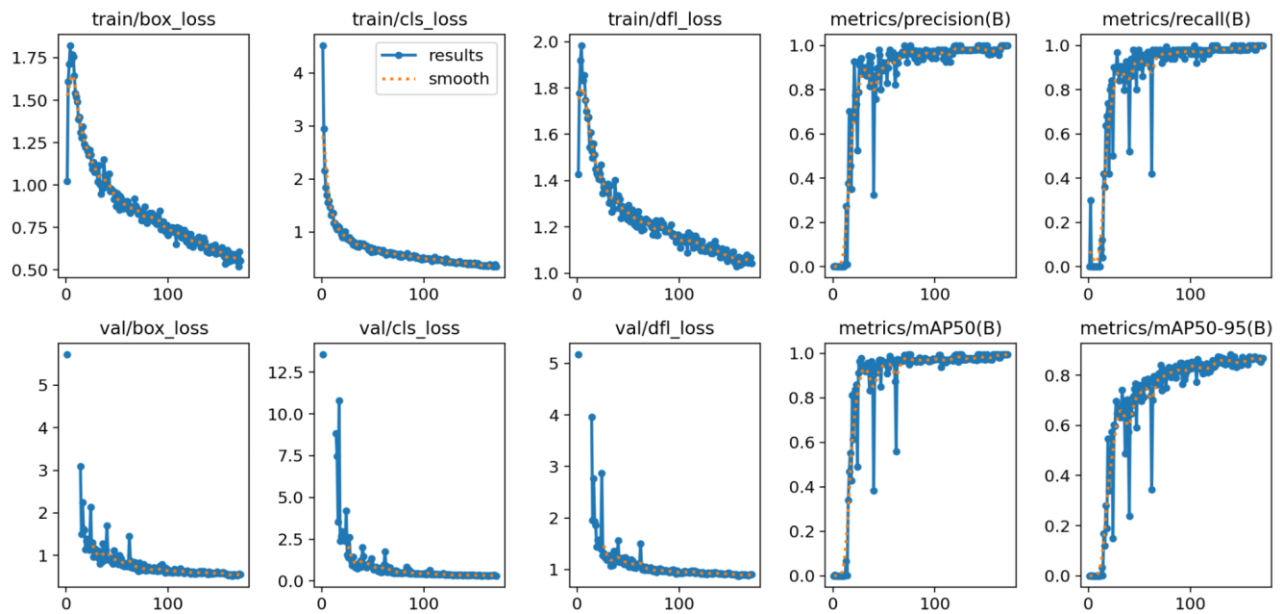
Validation Batch prediction:



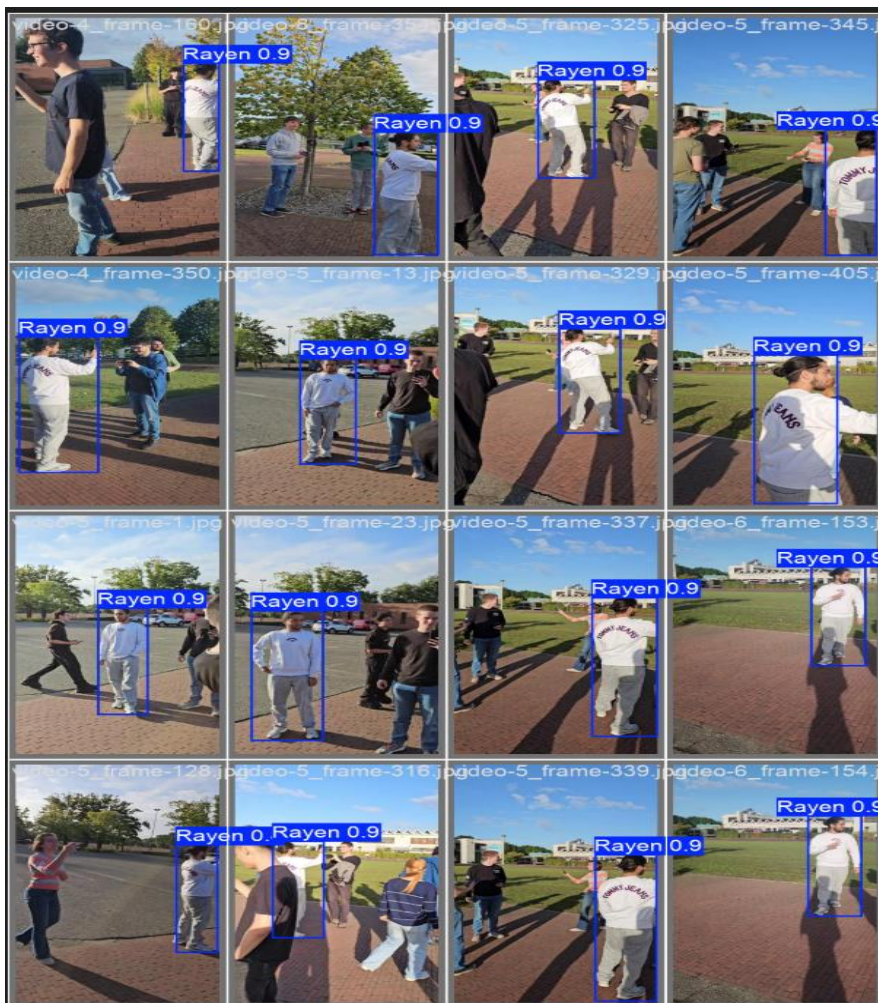
Conclusion for this model: This model is way better than when I started. You can also see that the results are way more stable (F1-score and precision). The biggest improvement here is the clean and improved dataset using script (2.3.5 YOLO editor). The dataset the early model was based on had a lot of incorrect data: “wrongly interpolated bounding boxes and wrong labels.” It was also smaller and unbalanced. Augmentation also helped a lot. While this model does very well on video data and the validation set it does poorly on the image test set compared to the validation set. My hypothesis is that the model has a difficult time training on much variation and many different angles. Also when we use videos the people are much further from the camera compared to when we take photos/selfies. The model is well-trained on the whole body of a person but when a person is much closer to the camera and only the face/part of the body is visible (most of the test set) my model fails. (Maybe I should have stuck to the image training set to be closer to the test set but using videos I got way more data and it is never the goal to get as close to perfect to the test set. The goal is to make a model that can generalize well.) I don't think changing the model from 11L to 11X is worth it. I tried the same improvements on a YOLO 11L model and got almost the same result. 11X was still a bit better but it's not 2 times better maybe a 5-12% improvement at best. But it is 2 times as big as 11L and inference/training is way heavier/slower. So the improvements do not outweigh the additional cost.

3.3.3 The Rayen set (yolo 11 X augmentation)

This model was made to see if it would help to train on a single person at a time. My thinking was that if I have fewer labels my model will train/focus more on that one label but the results show that there is no real improvement for Rayen compared to the previous model.



Validation Batch prediction:



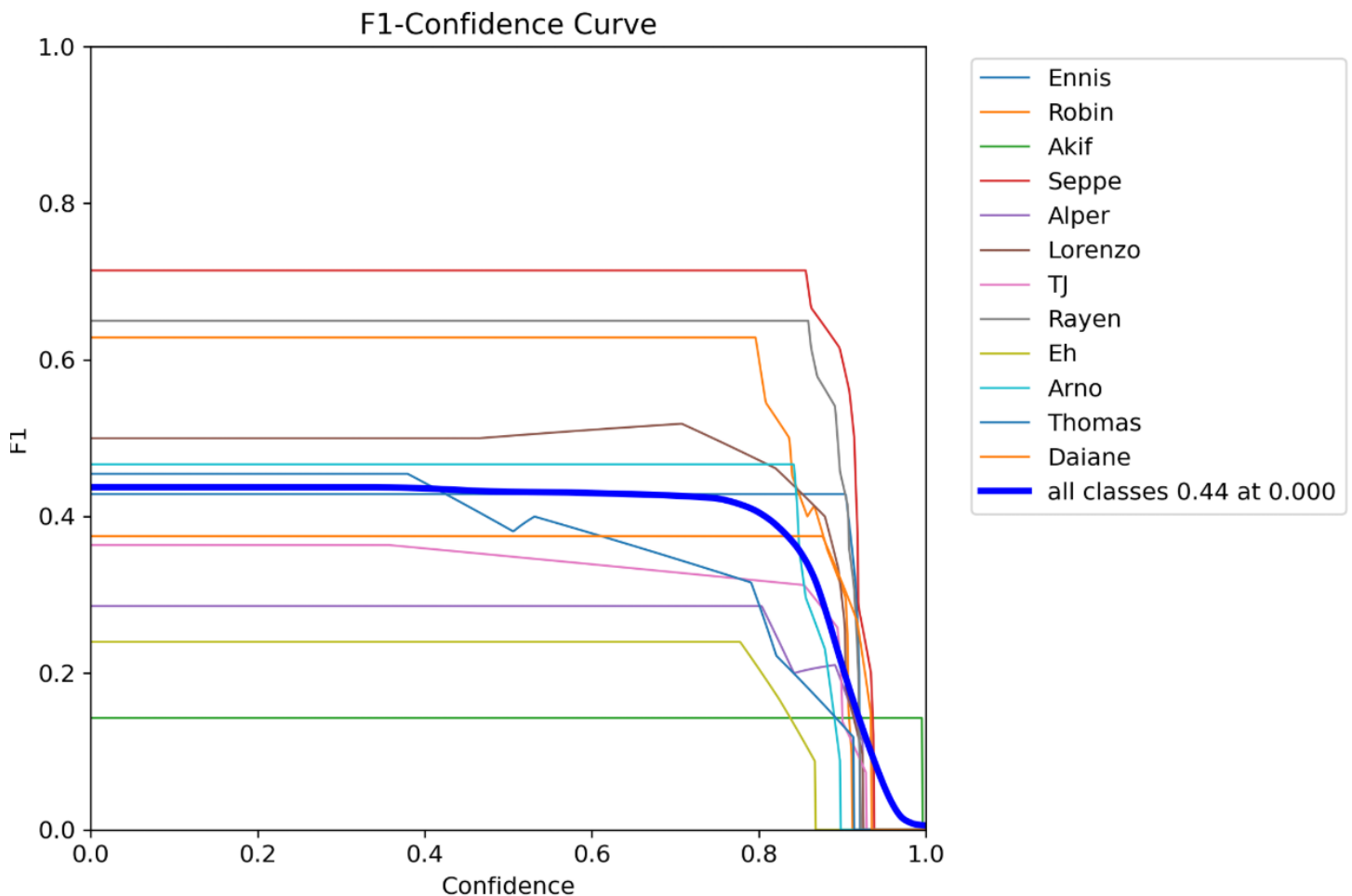
Conclusion for this model:

It works well but it is not an improvement from the previous model. Even if there were an improvement I would not go with it. It is too unrealistic to use. Doing this would mean training 11 different YOLO models which takes around 1.5 hours of time each and also having 11 different models for inference which is not realistic. If each model takes 15 seconds to make a prediction then $11 \times 15 \text{ s} = 165 \text{ s} \rightarrow 2 \text{ minuten } 45 \text{ seconden}$ to predict a single image to make a prediction on the whole test set(63 img's) i would take $165 \text{ s} \times 63 = 10\,395 \text{ s} \rightarrow 2 \text{ hours } 53 \text{ minuten}$ of inference. This is way too long so I did not pursue this approach. I went to face detection next.

3.3.4 yolo face detection (yolo 11 N agmettaions)

This was an additional model that I trained at the end of the project in the last two hours before the deadline). I didn't have much time and I wanted a small model because: A it's much faster to train, and B I wanted to compare it to the results of the big models. The conclusion was that the performance was very bad.

This model was trained on faces instead of the full bodies data set 4 (this was created with the same script as data set 3 but with a small modification: instead of extracting faces, I replaced the bounding box of the body with the bounding box of the face). Although this model was added to my final project, it does not really contribute anything.



Part 4: Face Detection

This section describes how faces are detected in images and classified using dataset3 and the InsightFace library.

This part does not contain the full inference pipeline; for this see Part (Final Project/inference) where the model implementation is described.

4.1 How it works

The idea is simple: we have a dataset of faces, we embed all of them, cluster the embeddings and label each cluster. When a new image is processed all faces are detected. For each face an embedding is created and compared to the stored embeddings. A vector distance metric (**cosine similarity / cosine distance**) is used to find the closest labeled embeddings which determines which cluster (person) the face belongs to.

How do we obtain the embedding dataset and how do we label it?

There are two approaches: **unsupervised learning** (best practice when no labels are available) and **my supervised approach** since I already had labeled data.

4.2 DeepFace

At the start I looked at **DeepFace** for face detection. DeepFace is a framework for face recognition detection and classification.

However I decided **not to use DeepFace** due to numerous dependency issues. It requires specific versions of TensorFlow and CUDA and I could not get it to run properly on my GPU. Even if I managed to make it work with that specific CUDA version PyTorch would not support it which meant I could not use my GPU for training my YOLO model. Because of all these compatibility problems I switched over to **InsightFace** which was more stable and easier to integrate into my pipeline.

4.3 InsightFace and buffalo_I

InsightFace is an open-source library for face analysis that provides tools for face detection alignment , embedding generation and face recognition. It is widely used because it offers reliable state of the art models that can be easily integrated into a face recognition pipeline.

buffalo_I is a preconfigured model package within InsightFace. In this project I only use it for face detection and face embeddings. The package combines multiple backbone models: a RetinaFace-based model for face detection and an ArcFace-style backbone for face feature extraction and embedding generation. These embeddings are optimized for similarity comparison using cosine similarity.

I also experimented with Antelope V2 InsightFace's newer and larger model. However it is significantly bigger showed only minor improvements in embedding quality for my use case and does not include a face detection/extraction model. Because of this buffalo_I was the more practical and efficient choice for this project.

4.4 Face dataset

First a face dataset (dataset3 see 2.1) is created. I used the InsightFace library with the **buffalo_L** model to detect faces. I already had a YOLO dataset where every bounding box was labeled. I wrote a script that detects a face and computes the center point of the detected face and checks which YOLO bounding box this point falls into using the corresponding YOLO **.txt** file. The face is then saved with the matching label (**Face_Robin_1.png**).

If a face is detected but its center point does not fall inside any YOLO bounding box it is ignored (background people).

This resulted in a dataset of about 3200 labeled face images. These faces are then embedded and clustered.

4.5 Embedding and clustering

The first step is to use an embedding model to generate embeddings for all face images in the dataset.

Because all faces in my dataset are already labeled I can directly assign the same labels to the embeddings. I do not need to discover clusters since each embedding already belongs to a known person.

In most real-world cases however you have a dataset with faces but no labels. In that case unsupervised learning is used to cluster the embeddings.

4.5.1 Unsupervised learning

This is the method used by others (Enis,Rayen.....).

First an unlabeled face dataset is embedded using a face embedding model using

(something like an ArcFace-based model).

Next an unsupervised clustering algorithm (such as k-means) is applied to the embeddings. The **elbow method** can be used to estimate the optimal number of clusters by finding a balance between cluster compactness and the number of clusters.

Finally the resulting clusters are manually or semi-automatically labeled.

4.5.2 Supervised learning (my approach)

Since I already had all faces labeled I did not need to use unsupervised learning. Each embedding is directly assigned to its corresponding person so the clusters are already defined by the existing labels.

I do note that **unsupervised learning is better in most cases**. It removes the need to manually label large amounts of data and makes it much easier to add new data or introduce new people. In that case you only need to re-cluster the embeddings instead of labeling everything again.

My approach is faster to set up and easier to implement when labeled data already exists. Using pre-labeled data also gives certainty that the embeddings are assigned to the correct person as long as the original labels are correct. With unsupervised learning two different people with very similar embeddings may end up in the same cluster.

I chose this method not because it is better in general but because it was the easiest option at the time since I already had pre-labeled data from my YOLO dataset.

4.6 experimentation

I experimented with different underlying models image sizes and image dimensions (RGB and black and white). A summary of these experiments is provided in the table below:

model	Img size	color
bufalo_l	640*640 px	rgb
bufalo_l	640*640 px	Black and white
bufalo_l	960*960 px	rgb
Antelope v2	640*640 px	rgb
Antelope v2	960*960 px	rgb

In the end I used **bufalo 640x640** in RGB. While black and white also yielded good results, I chose the RGB format to allow for the embedding of more data.

I experimented with embedding faces at a higher resolution of **960x960**, reasoning that this would allow for more information per face. However, this did not improve the results; in fact, the performance declined. This is likely due to the embedding models being trained on **640x640** images, so I maintained that resolution.

4.7 Embedding and Clustering Results

These are the results of the **bufalo_l 640x640 RGB embedding dataset**.

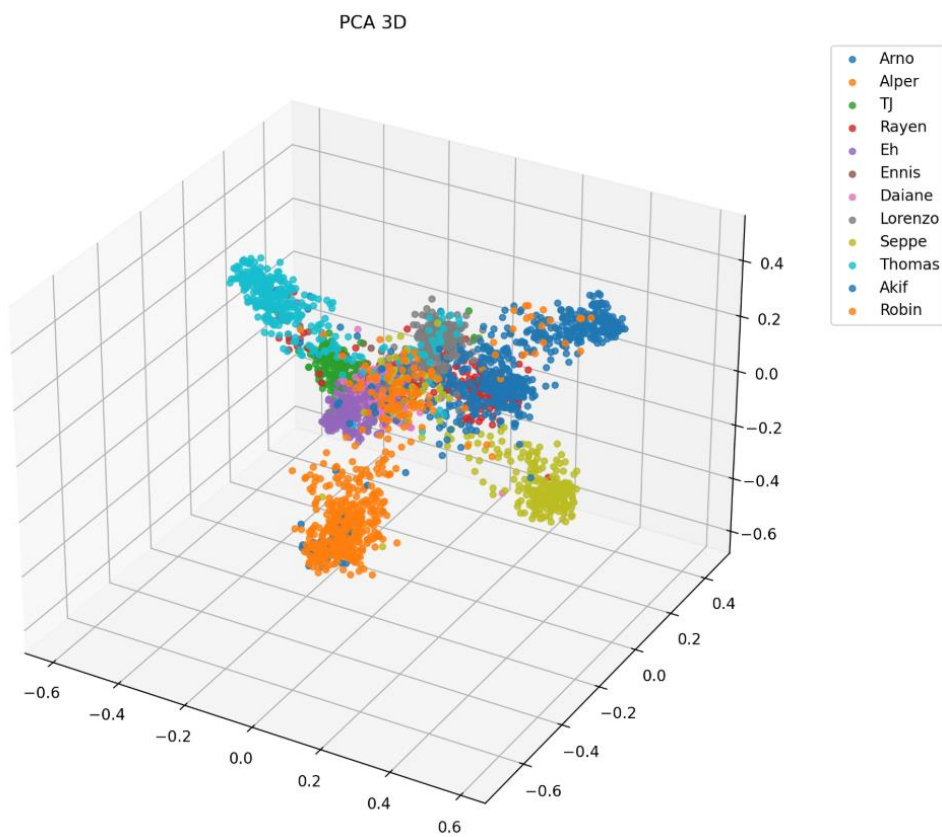
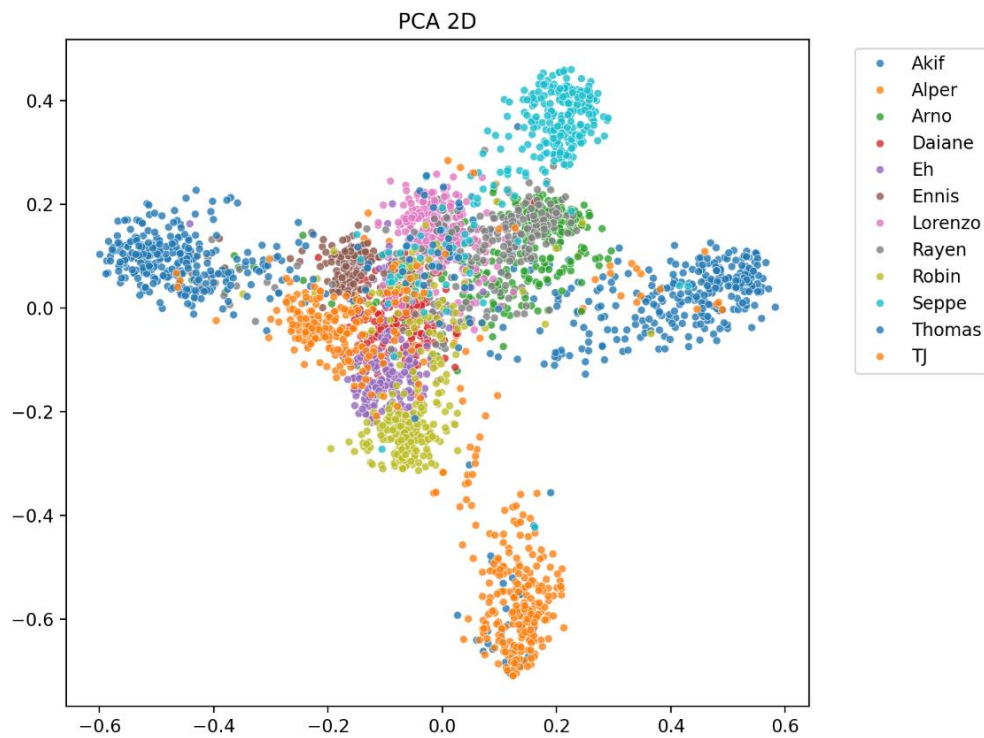
The images are embedded in **512 dimensions**, which cannot be directly visualized or plotted.

Therefore, dimensionality reduction is required to project the data into **2D or 3D space**. For this, **PCA, t-SNE, and UMAP** were used.

These are the results of my embeddings.

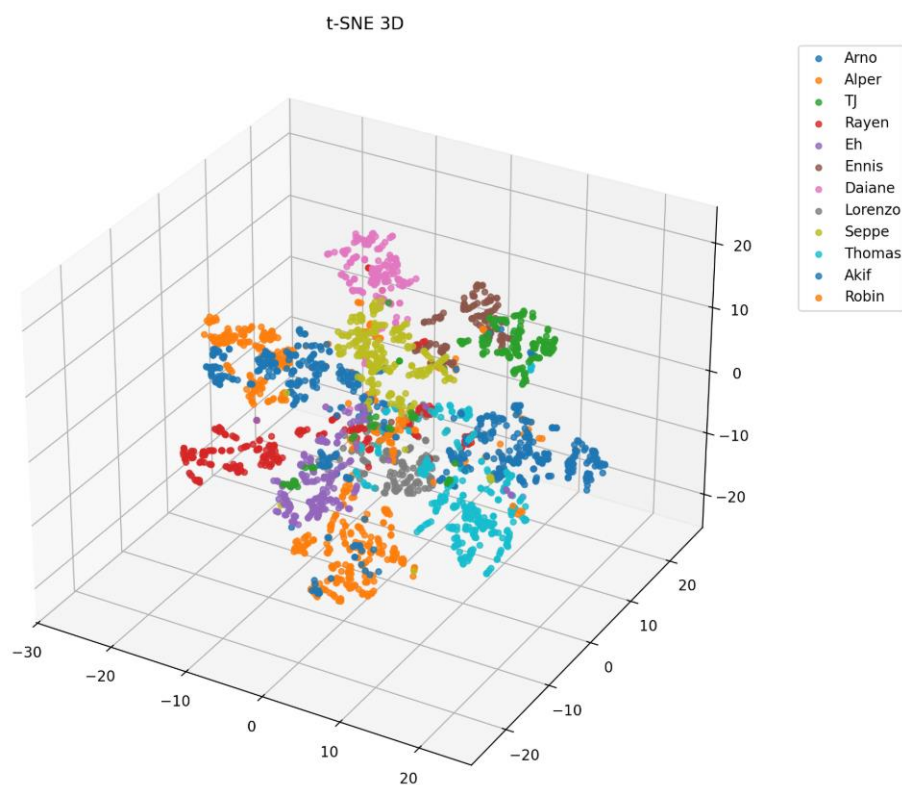
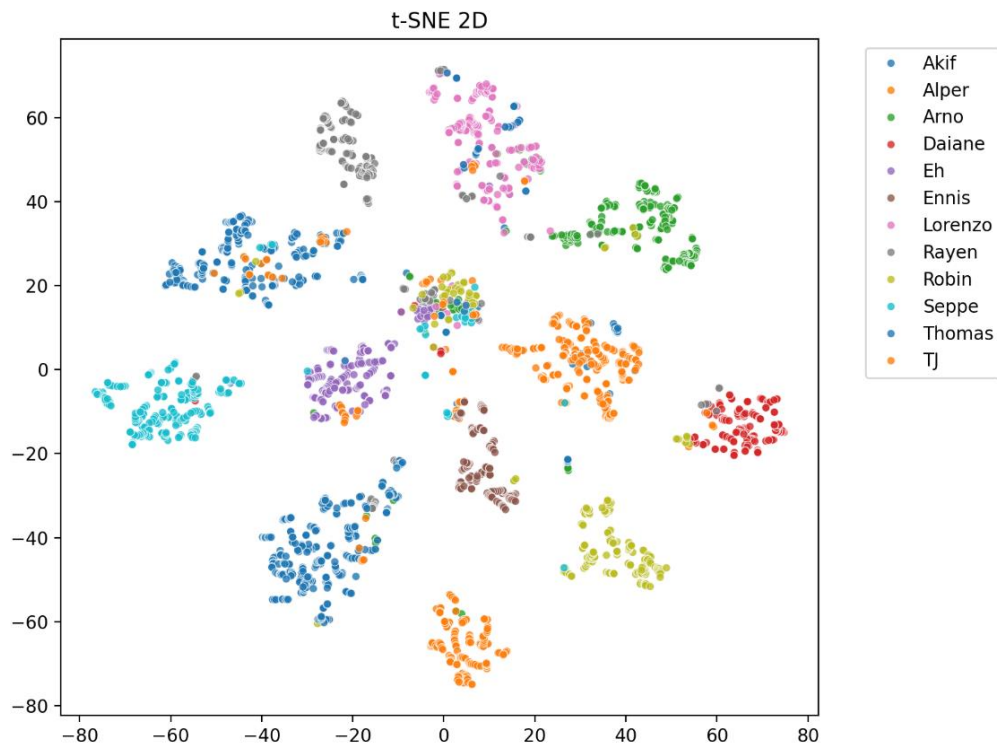
4.7.1 PCA (Principal Component Analysis)

PCA is a linear dimension reduction technique that projects the data onto directions with the highest variance. It is fast and simple, but it cannot capture non-linear relationships, which makes it less suitable for complex embedding structures.



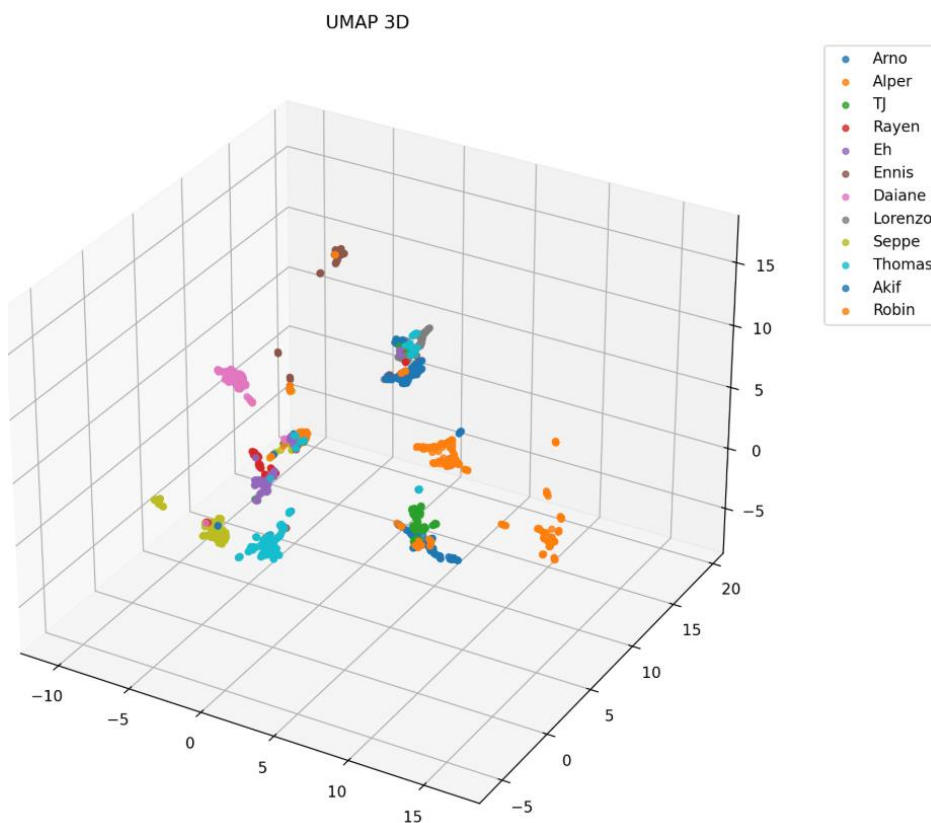
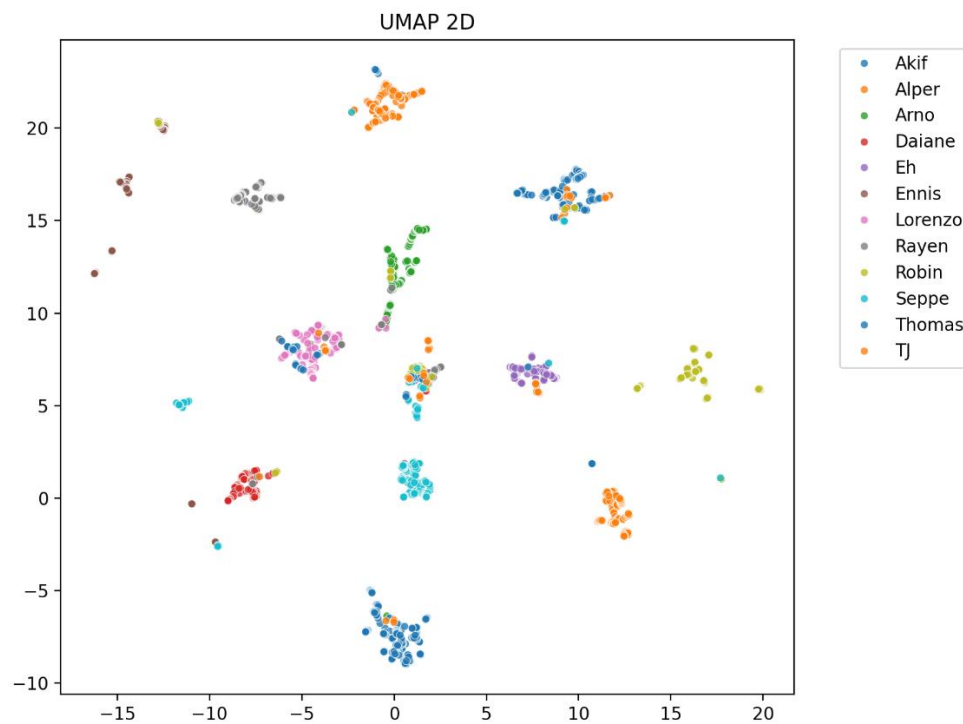
4.7.2 t-SNE

t-SNE is a non-linear technique that focuses on preserving local similarities between data points. It is very good for visualizing clusters, but it is slow, sensitive to hyperparameters, and global distances between clusters are not reliable.



4.7.3 UMAP

UMAP is a non-linear dimension reduction method based on manifold learning. It preserves local structure well while also keeping more global structure than t-SNE. It is faster and more stable, but the results still depend on parameter choices.



4.7.4 Conclusion

PCA is useful for a quick overview but misses complex patterns. t-SNE gives clear local clusters but weak global structure. UMAP offers the best balance between local and global structure and is the most practical for embedding visualization in my opinion.

Part 5 – Inference Code

Introduction

This section explains the complete inference pipeline used to recognize people in images. The pipeline combines multiple independent recognition methods and merges them into one final prediction list.

The main idea is:

No single model is perfect, but multiple imperfect models combined can produce a more reliable result.

5.1 Inference Pipeline

For every image, the following pipeline is executed:

1. Image scaling
2. Face detection/extraction
3. Embedding matching
4. Flipped image detection
5. YOLO detection
6. Prediction merging
7. Visualization and logging

Each step is implemented using dedicated functions.

Step 1 – Image preparation

```
scaled_img = scaleimg(img,2,True)
```

Function: `scaleimg(img, factor, upscale)`

Purpose:

- Resizes the image by a scale factor.
- Up/down scaling improves detection of small faces.

How it works:

- Computes new width and height
- Uses `cv2.resize`

This improves both InsightFace and YOLO detection accuracy.

Step 2 – Face detection

```
data = detectface(scaled_img)
```

Function: `detectface(img)`

Purpose:

- Detects faces using InsightFace.
- Extracts bounding boxes and embeddings.

Returned per face:

```
(cropped_face, x1, y1, x2, y2, embedding)
```

Important details:

- Faces are sorted from left to right.
- Each face contains a 512-D embedding vector.

This makes later matching possible.

Step 3 – Face recognition using embeddings

For each detected face:

```
pred_name, pred_score = find_person(face[5])
```

Face is a list and the 6 item in that list is the embedding of that face

Function: `find_person(embedding)`

Purpose:

- Matches a face embedding against stored embeddings.

Steps:

1. Normalize embedding using `l2_norm`
2. Load each `.npy` reference embedding
3. Compute cosine similarity (see 5.2)
4. Sort by similarity
5. Keep top N above threshold
6. Extract names from filenames
7. Select final identity using voting logic

Returned:

(name, similarity_score)

Supporting functions

`l2_norm(x)`

Normalizes vectors so cosine similarity becomes reliable.

`getmostcomonname(list)`

Chooses the most frequent predicted name.

If there is a tie:

- Calls `highest_average`

`highest_average(data, names)`

Chooses the name with the highest average similarity score.

This prevents one accidental high match from dominating.

Step 4 – Method 2: Flipped image detection

```
flipped_img = cv2.flip(scaled_img, 1)
data_flipped = detectface(flipped_img)
```

This repeats the entire embedding pipeline on a horizontally flipped image.

Why flipping?

Some faces are detected better from a mirrored view due to:

- Pose
 - Lighting
 - Partial occlusion
 - Model bias
-

Inverting bounding boxes

Because the image is flipped, bounding boxes are mirrored.

They must be converted back:

W is the width of the bounding box

```
new_x1 = w - x2
new_x2 = w - x1
```

This restores correct spatial positions.

Reversing prediction order

```
method2_preds.reverse()
```

Why this is required

When an image is flipped:

- Left becomes right
- Right becomes left

After restoring bounding box coordinates, the list order is still reversed.

Reversing the list restores **left-to-right face ordering**, which is required for correct merging with other methods .

Without this:

- Faces would be merged with wrong neighbors
- Overlap logic would fail

Step 5 – Method 3: YOLO detection

```
method3_preds = yolo_prediction(scaled_img)
```

Function: `yolo_prediction(img)`

Purpose:

- Runs YOLO model inference.
- Converts output into unified format:

[label, confidence, [x1, y1, x2, y2]]

YOLO acts as a **classification-based** method instead of embedding-based.

Step 6 – Merging predictions

```
final_list = merge_predictions([method1_preds, method2_preds, method3_preds])
```

Function: `merge_predictions(methods_list)`

This is the fusion stage.

Steps:

1. Flatten all predictions
2. Remove predictions below confidence threshold
3. Sort by x-coordinate
4. Resolve overlapping boxes:
 - If overlap is large then keep highest confidence prediction
5. Remove adjacent duplicate labels

This produces one clean prediction per face.

Step 7 – Drawing and logging

`drawOnimg(data, img_path, scaled_img, predictions)`

Function: `drawOnimg(...)`

Purpose:

- Draw bounding boxes
 - Draw name
 - Draw confidence score
 - Save output image
-

5.1.1 Logging and CSV export

`Log(...)`

`append_kaggle_csv(...)`

The `log` function creates a file for every attempt made and logs what happens to each image.

I log the result of each method and the final result, as well as the parameters for the function `find_more_faces` (this was a function I tried to find more faces with, but it did not improve results). I used the log data to see the effect of each model and to see the impact between logs when I made changes.

The `append_kaggle_csv` function is used to add the predictions to a CSV file that can be used in the Kaggle competition.

5.1.2 Find more faces

This is a function I created before I started combining models.

The idea was as follows: first, I run a normal prediction to detect faces without classification. For example, if five faces are found, I then manipulate the image in a loop. I rotate the image by five degrees and check whether more faces are detected. If so, I save that prediction. I then rotate the image by another five degrees and repeat this process until the loop finishes. After that, I flip the image and run the same loop again, this time on the flipped image. There was also an option to convert the image to grayscale after the first loop and then repeat the entire process. This was done to evaluate how much grayscale conversion would influence face detection.

The parameters of this function determine how many steps are used in the loop, the degree change per step, and two booleans: one for whether the image is flipped and one for whether the image is converted to black and white.

This function was primarily created for experimentation, to evaluate whether image manipulations such as rotation, flipping, and grayscale conversion would improve face detection. I found that image rotation had no real impact on the number of faces detected. Occasionally, an extra face was found, but there was no meaningful improvement, and the angles at which faces were detected were inconsistent. Testing every angle in steps across all images would also make the final inference pipeline too slow. For grayscale images, I did not find a real difference; the results were almost identical to RGB images.

Image flipping, however, produced more interesting results. It did not detect more faces overall, but it did detect faces that were missed in the original image. At the same time, some faces detected in the original image were missed in the flipped version (see section 5.4 for an example). This observation led to the idea of combining different methods in the final prediction pipeline.

5.2 Cosine Similarity

Cosine similarity measures the angle between two vectors:

$$\frac{A \cdot B}{||A|| ||B||}$$

Why it is used:

- Independent of vector magnitude
- Ideal for embedding comparison
- Stable for face recognition

Because the embeddings are normalized, cosine similarity becomes a direct identity similarity score.

5.3 Combining Different Methods

After experimenting with the *find more faces* function (5.1.2), I noticed that in the flipped version some faces were found that were not detected in the original image. However, the mirrored version also missed some faces that were originally found (see 5.4 for an example).

I created three different methods for finding faces:

Method 1: normal

Method 2: flipped

Method 3: YOLO

Each method returns a list of predictions. These predictions are then added to a list called *methods*. I designed it this way so I can later add more methods if needed; each new method only has to add its predictions to that list.

That list is then passed to the function that combines all predictions.

The end result is

- fewer false or incorrect predictions.
- Find and classify more faces.
- Improved confidence.

5.4 Combined Methods Results

Methode 1 face detection no flip



rayen

thomas

robin

lorenzo

tj

As we can see this works very well but the face of arno is clearly visible but not detected

Methode 2 face detection flip



rayen thomas arno robin tj

Here we got arno but lost Lorenzo

Methode 3 face detection yolo



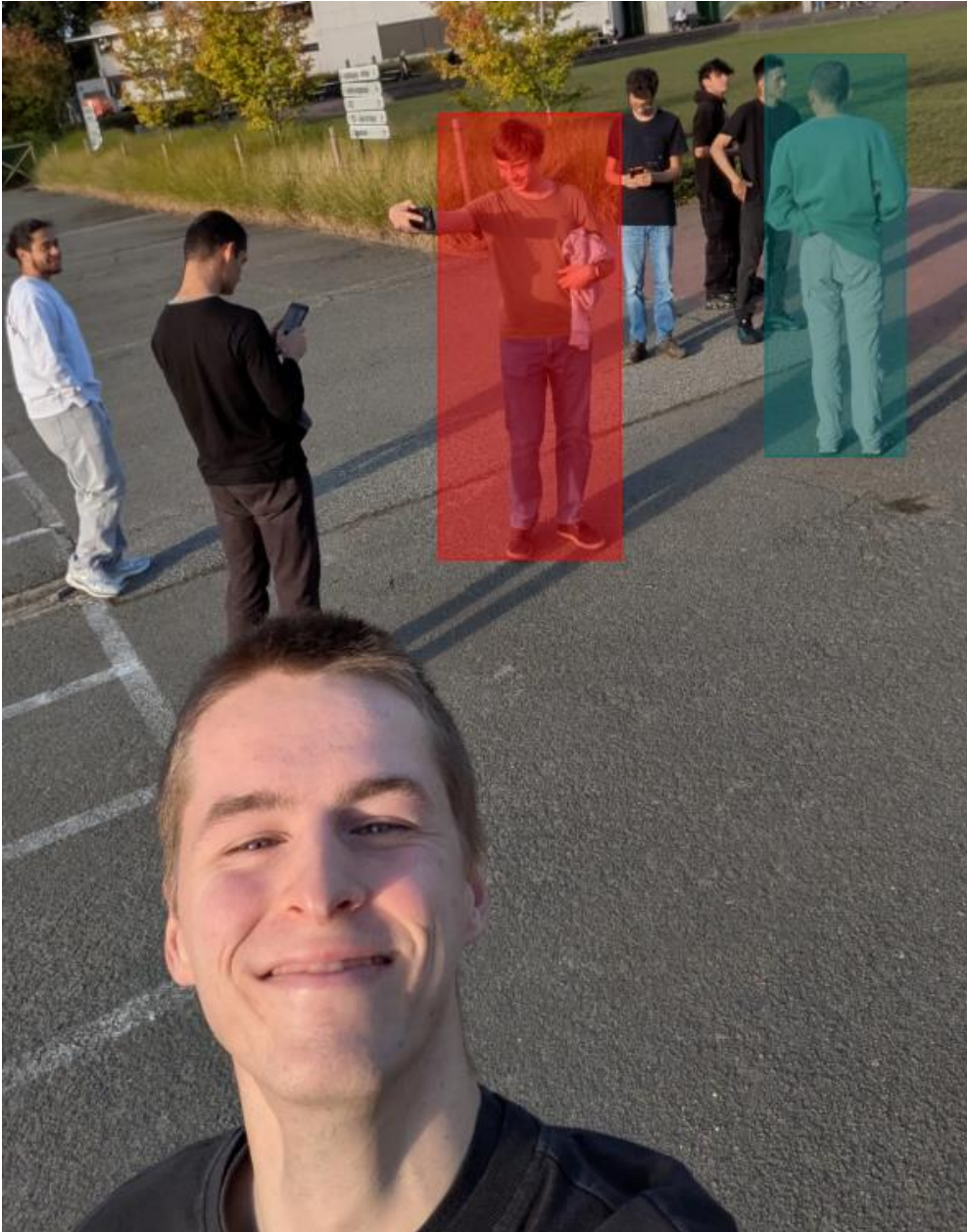
tj

Yeah, the YOLO face detection is really bad; it only got TJ, but at least it was a correct detection.

The face YOLO model was a last-minute addition a few hours before the deadline.

Here is also the result of my YOLO model for full body. It is not much better on this example.

Methode 3 full body detection yolo



akif alper

The yolo model's performance is really inconsistent. While it yields nice results for certain images within the test set, it completely fails on others, as is clearly demonstrated here. :(

Here are the combined prediction results:

Combining the methods result



rayen thomas arno robin lorenzo tj

Here, almost everybody is found, and the results from methods 1, 2, and 3 are combined (Lorenzo's method 1 and Arno's method 2) to make the best prediction so far. Perhaps it would be interesting to make another model trained on the backs of people so we can also find Enis and Seppe. The reason they are not found is because their faces are not visible.

Part 6 – GUI and Masking Clarification

After the project was done, I created a **GUI (web app)** to make the system easier to use. The user can upload an image, select which models to use, and then view the predictions.

At first, the system only produced **face predictions**. I was not happy with that, because it only shows faces and not the full person.

So I added **person segmentation** using **SAM3 from Meta**.

Now the system does not only draw face boxes, but also creates **segmentation masks for each person** in the image.

6.1 Gradio

What is Gradio?

Gradio is a Python library that makes it easy to build a **web interface for machine learning models**. It connects Python functions directly to UI elements like image upload, buttons, and checkboxes.

Why it is easy to use

Gradio is easy because:

- No HTML or JavaScript is needed.
- Python functions are directly linked to the interface.
- It automatically shows images and results.
- It is very fast to set up and test models.

Main concept

The main idea is simple:

You write a Python function, and Gradio creates a web interface around it.

The input from the user goes into the function, and the output is shown in the browser.

6.2 Segment People with SAM3

What is SAM?

SAM stands for **Segment Anything Model** and is developed by Meta.

It is a **foundation segmentation model**, which means it can segment almost any object without being trained for that specific object.

SAM3 is the newest version and gives better results and performance.

What type of model is it?

SAM is based on **vision transformers** and is trained on a very large segmentation dataset. It does not classify objects, it only finds **where objects are** in the image.

How it works in this project

SAM3 takes:

- An image
- A text prompt, in this case: "person"

And returns:

- A segmentation mask for each person
- A bounding box for each person

SAM knows where the people are, but it does not know who they are.

[Image with SAM results here]

6.3 SAM3 Implementation and Pipeline

The pipeline has three main parts:

1. Frontend (Gradio GUI)
 2. Face classification predictor
 3. SAM3 segmentation module
-

Pipeline

Step 1 – User input

The user uploads an image and selects which models to use.
Then the user clicks **"Go to next step"**.

Step 2 – Face prediction

The frontend sends the image and selected models to the face predictor.

The predictor runs the pipeline from Part 5.1 and returns:

```
[[[x1,y1,x2,y2], "robin"], ([x1,y1,x2,y2], "enis"), ...]
```

This list is saved as **face predictions**.

Step 3 – SAM3 segmentation

The same image is given to SAM3 with the text prompt **"person"**.

SAM3 returns:

- Person masks
- Person bounding boxes

Note that:

- SAM3 does not return labels.
- The order of the detections is not left-to-right.

Because of this, the person boxes are sorted based on the x-coordinate of their center point.

Final goal

We want a list like:

```
[(mask,"robin"), (mask,"enis"),, ...]
```

Combining face and SAM results

We now have two lists:

Face list

```
[[[x1,y1,x2,y2], "name")....]
```

SAM list

```
[(mask, [x1,y1,x2,y2])....]
```

For each face:

1. We calculate the center of the face box.
2. We check in which person box this point falls.
3. That person mask gets the face label.

If a person box does not contain any face, it is labeled as:

PersonX

This usually happens for background people.

6.3.1 Code Explanation

Here i will go over the main function

the code uses

SAM3 setup

Creates the SAM3 segmentation predictor.

```
predictor = SAM3SemanticPredictor(overrides=overrides)
```

calCenter

Calculates the center point of a bounding box.

genClasificationPoints

Converts face predictions into center points with labels.

boxData

Extracts SAM bounding boxes from the SAM output.

maskData

Extracts SAM masks and converts them into binary masks.

sortData

Sorts persons from left to right based on bounding box position.

genNames

Matches face center points with SAM boxes and assigns labels.
If no match is found, the person is named **PersonX**.

makeResult

Creates the final result structure:

[image, [(mask,label), ...]]

maskProdiction

This function runs the full SAM pipeline:

1. Generate face center points
2. Run SAM3 with "person"
3. Extract boxes and masks
4. Sort them
5. Match faces to persons
6. Return labeled masks

The output is saved as **labeled masks**.

6.4 Results and GUI Usage

Step 1 – Upload image

1☐

2☐

#input image

Select people

give a input img

Image

Drop Image Here

- or -

Click to Upload

📁

📷

📄

select the methodes you want to use for clasification

☒ base face detection

☐ mirror face dection
use the base face face detection on a mirrored img to find more faces

☐ yolo face detection
uses yolo to to dectect more faces

all the selected prodiction methodes are combind for a final prodiction

go to next step

Step 2 – Select models

The user selects:

- Face prediction
- Mirrored face prediction
- YOLO

These predictions are combined into one final face result.

select the methodes you want to use for clasification

☒ base face detection

☐ m
use the

all the selected prodiction methodes are combind for a final prodiction

Step 3 – Production page

After clicking **Go to next step**, the user sees:

- Face bounding boxes with labels
- Person segmentation masks with labels

select the peapole you want to remove



go to Step 1

GUI note

The GUI/frontend(gradio) code is not explained because it is only used to make the system easier to use.

The main focus is the **combination of face classification and SAM3 person segmentation**, which produces labeled person masks from one image.

Part 7: Conclusion and Future Work

Conclusion

I am very happy with the end result and the final project. The predictions worked better than I expected.

I learned a lot, especially that cleaning and creating datasets takes the most time in a project. I also learned that face recognition works better than YOLO for this task. Looking back, I am happy with the final result.

However, there are some things I would do differently next time, and also some methods I would like to try that I did not get to in this project.

Things I Would Have Done Differently

Next time I create my datasets, I will focus more on the quality of the data and how much each data point really contributes. This time I made the dataset as large as possible (around 3200 data points per dataset), thinking that more data is always better. Sometimes it is, but even though my dataset is clean and correctly labeled, there is probably a lot of duplication because many frames are very close to each other.

For example, I know ENSIS has a dataset of around 400 data points and it works as well as mine. Of course, this is hard to compare because our test set is only 63 images. Still, 3000 embeddings is clearly overkill. I would like to find the sweet spot by starting with fewer, less repetitive data points and then adding more over time to see if the model actually improves.

At some point we talk about diminishing returns, where adding more data does not improve the model anymore. I think having more data is never a real problem, but it is not always efficient.

Now that I know I can use SAM3, I would also like to train a YOLO segmentation model and compare it with the same model using bounding box data on the same images. I want to see if segmentation improves detections and predictions. I expect it might, because segmentation has almost no background noise (the mask is the whole image), but I am not completely sure.

Lastly, I would like to try building and training my own CNN from scratch, maybe as a summer project.

For image recognition, there are also many models based on transformers. I did not look into this too much, but it would be interesting to try.